Thinking in English, Paying Less? Token Cost and Accuracy of English Pivoting for Arabic LLM Queries

Oussema Ben Elhadj
DGIMA, ENIS
Sfax, Tunisia
oussema.benelhadj@enis.tn

Abstract—Large language models are billed and scheduled per token, and the same Arabic text needs more tokens than its English translation. We measure this token tax on 12 tokenizers and find that it has fallen from 3.0–3.4× on older vocabularies to 1.2–1.7× on current ones, while an Arabic-centric vocabulary reaches parity (0.97×). We then test an English pivot on five models, three open and two served through an API: Qwen2.5-7B, Llama-3.1-8B, Qwen2.5-14B, Phi-4 and GPT-4.1. The saving a pivot delivers tracks the tokenizer closely. Phi-4, whose vocabulary charges Arabic 3.04×, saves 57–63% of prompt tokens, close to the 67% its tokenizer allows; GPT-4.1 saves 20–28% against a ceiling of 27.5%. The accuracy cost behaves differently. On 1,500 native Arabic exam questions the pivot loses 8.6 points for Llama-3.1, 12.4 for Phi-4, 16.1 for Qwen2.5-7B, 20.3 for Qwen2.5-14B and 24.8 for GPT-4.1: the stronger a model's Arabic, the more pivoting costs it, without exception across the five. Replacing the 600M translator with NLLB-200 3.3B, 5.5 times larger, recovers a median of 16% of the distance to a human English reference, so the loss is not mainly a matter of translator capacity. Per subject the damage is predictable and replicates across models (Spearman 0.58–0.76): Arabic grammar questions fall to 24%, near the 25% chance level, because translation destroys what they test, while computer science improves. In step-by-step mode GPT-4.1 saves 0.2% of weighted tokens and loses 13.2 points, since its English reasoning runs longer than its Arabic. Pivoting is a model- and content-dependent choice, benchmarks translated from English overstate its benefit, and a better translator is not the fix it appears to be.

Index Terms—token efficiency, tokenization, Arabic NLP, machine translation, large language models, inference cost

I. Introduction

Commercial LLM services charge per input and output token, and self-hosted models spend compute and memory in proportion to sequence length [1], [2]. Because subword vocabularies [3] are learned mostly from English text, other languages are split into more tokens for the same content. Users of those languages pay more for the same service and fit less text into the same context window [4]–[7].

Arabic has hundreds of millions of speakers. A natural engineering response to the token tax is to pivot: translate the Arabic query into English, answer in English, and return the result. Prior work shows that answering in English can improve accuracy [8], [9] and that translation or transliteration can cut GPT-4 costs for Indic languages [10]. Three questions remain open for Arabic. How large is the tax on current tokenizers? Does a pivot that costs no paid tokens keep accuracy on content that was not originally written in English? And if it does not, would a better translator fix it?

This paper makes five contributions:

- A measurement of the Arabic token tax on 12 tokenizers, from Llama-2 to GPT-4o, on 488 parallel passages (Section IV-A).
- A controlled comparison of four strategies (asking in Arabic, local machine-translation pivot, LLM self-translation, and a human-English upper bound) on five models spanning 7B to a frontier API model, in two answer modes, measuring tokens, accuracy and GPU time (Sections IV-B and IV-C).
- Evidence that the accuracy cost of pivoting rises with the model's Arabic ability, and that the token saving rises with the tokenizer's Arabic tax. The two move in opposite directions across our five models.
- A test of the obvious remedy. NLLB-200 3.3B in place of the 600M distilled model recovers a median 16% of the gap to human English (Section IV-D), which contradicts what the upper bound alone suggests.
- An evaluation of AI-free routing, in which ordinary code decides whether to pivot. Surface checks fail, a per-model calibration succeeds, and two answer-level methods work but cost more than they save (Section IV-H).

II. Related Work

Tokenization disparity. Petrov et al. [4] and Ahia et al. [5] showed that commercial tokenizers charge some languages several times more than English for parallel text. Solatorio et al. [6] linked the disparity to socio-economic inequality, and Lundin et al. [7] found that higher tokens-per-word predicts lower accuracy on African languages. Arabic-centric models such as ALLaM [11] extend the vocabulary to reduce the gap. We extend these measurements to current open and commercial tokenizers for Modern Standard Arabic, and we check them against token counts reported by two production APIs.

Answering through English. Translate-test, meaning translating the input to English before inference, is a long-standing baseline. Etxaniz et al. [8] showed that a model translating its own input outperforms direct inference, and cross-lingual-thought prompting [9] narrows gaps between languages. These works measure accuracy, not cost. Closest to ours, Nag et al. [10] reduced GPT-4 cost for 15 Indic languages by up to 90% through translation, transliteration and code-mixing. We study Arabic, use a free local translator so translation costs no paid tokens, measure reasoning length separately, test native-language content in addition to translated benchmarks, and vary the translator to see whether its quality is the binding constraint.

Token minimization. Other approaches compress prompts [12], budget reasoning tokens [13], or route queries to cheaper models [14], [15]. Work on tool-making and self-debugging [16], [17] replaces repeated LLM calls with reusable code and tests. Our routing experiment asks whether such AI-free code can decide when a pivot is safe.

III. Method

A. Cost model

Let Q, C, R, A be the English token counts of a question, its context, the model's reasoning and its answer, and let r be the ratio of Arabic to English tokens for the same content. With input and output prices p_{in}, p_{out}, asking in Arabic costs

$$\text{cost}_{ar} \approx p_{in}\, r(Q + C) + p_{out}\, r(R + A), \tag{1}$$

while a pivot whose translation runs locally costs

$$\text{cost}_{pivot} \approx p_{in}(Q + C) + p_{out}(R + A), \tag{2}$$

plus local translation time. The maximum relative saving is $1 - 1/r$. If the LLM translates instead, the translation itself is billed, and output tokens usually cost several times more than input tokens, so a self-translated pivot can cost more than asking directly. We test both cases.

B. Pipeline

The pivot pipeline uses a single paid step. The Arabic text is split into sentences with a rule-based splitter, each sentence is translated to English by NLLB-200 [18] on the local GPU, and the English question is sent to the LLM. Everything except the last step uses ordinary code or a free local model.

Two translation variants let us separate sentence-level from discourse-level errors. The segment variant translates each sentence independently, as above. The context variant translates the whole passage in one call when it fits in 450 tokens, and translates the question together with its options as a single numbered block, falling back to segments when the block does not parse back into the right number of options.

C. Strategies

- ar_direct: Arabic content with Arabic instructions, which is what users do now.
- ar_mt_pivot: NLLB translation, English instructions.
- ar_self_translate: the LLM first translates the Arabic block, then answers the translation; tokens of both calls are counted.
- en_human: the benchmark's human English version, an upper bound representing perfect translation.

Each strategy is run in two modes: number only, where the model replies with the option number in at most 16 new tokens, and step by step, with brief chain-of-thought reasoning [19] in the prompt's language followed by an answer line, in at most 512 new tokens.

D. Data

Belebele [20] has 900 multiple-choice reading-comprehension questions on 488 FLORES passages, with human-translated, aligned Arabic (arb_Arab), English and French versions. FLORES passages originate in English. Step-by-step mode uses the first 300 questions. ArabicMMLU [21] has native Arabic school and university exam questions; we sample 1,500, which is 300 per subject group, with seed 0. ArabicMMLU has no human English version, so only ar_direct and ar_mt_pivot apply there.

E. Models and setup

Qwen2.5-7B-Instruct [22], Llama-3.1-8B-Instruct [23] and Qwen2.5-14B-Instruct run with greedy decoding on two NVIDIA T4 GPUs (Kaggle); the 14B model is quantized to 4-bit NF4. Phi-4 and GPT-4.1 are queried through Azure AI Foundry at temperature 0. Token counts for the open models use each model's own tokenizer and include the chat template; for the API models we use the token counts the API itself reports. The token-tax study additionally uses GPT-4 (cl100k) and GPT-4o (o200k) vocabularies via tiktoken [24].

Answers are parsed by regular expressions, and unparsable answers count as wrong. In number-only mode they are at most 3.3% for all strategies except self-translation, which reaches 7.7%. In step-by-step mode the answer is taken from the final answer marker, which is Answer: N for English prompts and the Arabic equivalent for Arabic prompts; 892 of 896 GPT-4.1 reasoning outputs contain one. Differences are paired per question and reported with 95% bootstrap confidence intervals and exact McNemar tests. Translation quality is chrF [25] against the human English, computed with sacreBLEU [26]. To express a single cost, weighted tokens count output tokens 4×, a typical API price ratio; Section IV-G reports how much that choice matters.

TABLE I
Belebele, number-only mode: accuracy (%) and change in prompt tokens per question. $n = 900$. Pivot is the 600M segment variant.

Model	Arabic	Pivot	Human	Tok. Δ
Qwen2.5-7B	84.4	77.1	92.6	−29.4%
Llama-3.1-8B	69.3	72.8	90.1	−29.9%
Qwen2.5-14B	90.2	78.4	94.7	−29.4%
Phi-4	90.4	80.8	94.4	−62.6%
GPT-4.1	93.5	84.9	95.9	−19.7%

TABLE II
ArabicMMLU, number-only mode: accuracy (%) with the 600M segment pivot. $n = 1{,}500$. Models ordered by Arabic accuracy.

Model	Arabic	Pivot	Δ
Llama-3.1-8B	54.6	46.0	−8.6
Phi-4	62.1	49.7	−12.4
Qwen2.5-7B	63.1	47.0	−16.1
Qwen2.5-14B	66.3	46.1	−20.3
GPT-4.1	80.6	55.8	−24.8

IV. Results

A. The Arabic token tax

Fig. 1 shows the ratio of Arabic to English tokens on the 488 parallel passages. Older English-centric vocabularies need 3.0–3.4× as many tokens (GPT-4 cl100k 3.04, Llama-2 and Phi-3.5 3.42). Current general-purpose vocabularies need 1.2–1.7×: GPT-4o 1.38, Gemma-3 1.47, Qwen2.5 and Qwen3 1.63, Llama-3 1.66. The Arabic-centric ALLaM tokenizer needs 0.97×. On current tokenizers Arabic's tax is close to French's, 1.66 against 1.59 on Llama-3.

The two API models let us check this on real prompts rather than on parallel passages. On the same 900 Belebele questions, the English prompts cost Phi-4 178.8 tokens and GPT-4.1 177.4, a difference of less than 1%. The Arabic prompts cost 479.5 and 221.5. Two tokenizers that agree on English diverge by a factor of 2.17 on the same Arabic text.

B. Cost and accuracy on Belebele

Table I summarizes the number-only results. The local-translation pivot cuts prompt tokens by 20–63% depending on the tokenizer, and the size of that cut follows the tax in Fig. 1 closely. Phi-4, at 3.04×, saves 62.6%, against a theoretical ceiling of 67%. GPT-4.1, at 1.38×, saves 19.7% against a ceiling of 27.5%. Equation (2) therefore predicts behaviour on two models it was not fitted to.

Accuracy depends on the model. For Llama-3.1, the weakest in Arabic, the pivot raises accuracy by 3.4 points (95% CI [+0.1, +6.9], $p = 0.055$) in number-only mode and by 17.7 points ([+10.7, +24.3], $p < 0.001$) in step-by-step mode, where Arabic reasoning reaches only 58.7%. Every other model loses: 7.4 points for Qwen2.5-7B, 11.8 for Qwen2.5-14B, 9.5 for Phi-4 and 8.6 for GPT-4.1, all with $p < 10^{-11}$ except the first.

Self-translation is the least accurate strategy in all four settings where we ran it, 10.0 to 24.3 points below asking in Arabic, and costs 32–185% more weighted tokens.

The human-English upper bound beats Arabic for every model: by 8.1 points for Qwen2.5-7B, 20.8 for Llama-3.1, 4.5 for Qwen2.5-14B, 4.0 for Phi-4 and 2.3 for GPT-4.1, at the pivot's token cost. The margin shrinks as the model's Arabic improves, which is what the last row implies: at 93.5% Arabic accuracy there is little headroom left for any change of language to exploit.

C. Native Arabic content

Belebele's Arabic is itself a translation from English, which may make translating it back unusually easy. On native ArabicMMLU questions the pivot still saves 26–57% of tokens, and every model loses accuracy. Table II orders the five by their Arabic score. The loss grows with that score at every step, from 8.6 points for the weakest model to 24.8 for the strongest, with no exception.

The pivot column is the more useful one. The five models span 26 points when they read Arabic and 9.8 points when they read the pivot's English. A model 26 points better at Arabic ends up less than 10 points better after translation, so translation quality imposes a ceiling that erases most of the difference between models. Buying a better model and then pivoting spends money on an advantage the pivot removes.

The loss is not spread evenly over subjects, and the pattern is stable across models. Ranking the 17 ArabicMMLU subjects by how much the pivot costs gives Spearman correlations of 0.58 to 0.76 between every pair of the three open models. Arabic grammar is the worst subject for all three, at −47.0, −27.3 and −53.0 points; pooled, it falls to about 24%, which is the chance level for a four-option question. A question asking which form is grammatically correct in Arabic stops being that question once it is in English, and no translator can help. Computer science is the only subject that improves for two of the three models, by up to 9.2 points, which fits a field whose terminology is English anyway. By group, the ordering is Language −36, Other −21, Social Science −14, STEM −10 and Humanities −10.

A metric that divides cost by the number of correct answers hides this. On ArabicMMLU the pivot needs 7–15% fewer weighted tokens per correct answer while answering 9–16 points fewer questions correctly. Efficiency claims should report accuracy separately.

An accident of the API runs makes the same point. Azure's content filter refused 20 of GPT-4.1's 5,700 prompts. All eight ArabicMMLU refusals were in the pivot condition and all eight were Arabic Language (General) questions; the Arabic originals passed. The machine translation turned these items into English that a safety classifier rejected.

TABLE III
Replacing NLLB 600M with NLLB 3.3B, context variant. Closed is the gain as a fraction of the distance to human English (Belebele) or to the Arabic baseline (ArabicMMLU).

Model	600M	3.3B	Δ	Closed
Belebele				
Qwen2.5-7B	75.7	75.9	+0.2	1%
Llama-3.1-8B	72.4	74.7	+2.2	13%
Qwen2.5-14B	77.9	81.1	+3.2	19%
ArabicMMLU				
Qwen2.5-7B	49.3	53.7	+4.4	32%
Llama-3.1-8B	47.9	50.7	+2.7	41%
Qwen2.5-14B	48.0	50.8	+2.8	15%

D. Does a larger translator help?

The gap between the pivot and human English invites an obvious remedy: use a better translator. We tested it by replacing NLLB-200 distilled 600M with NLLB-200 3.3B, 5.5 times the parameters, on the same questions and the same pipeline. Translating the full set took 2,616 seconds against 537.

Table III gives the result. The gains run from 0.2 to 4.4 points, and four of the ten comparisons are not significant at the 5% level. Expressed as a fraction of the distance to the ceiling, which is human English on Belebele and the Arabic baseline on ArabicMMLU, the 3.3B model recovers between 1% and 41%, with a median of 16%. Qwen2.5-14B on Belebele moves from 77.9 to 81.1 while human English sits at 94.7.

So the pivot's accuracy cost is mostly not a translator-capacity problem. This contradicts the reading that the human-English bound invites, and it is consistent with the subject analysis: a translator of any size will not preserve a question about Arabic morphology.

One secondary finding is worth keeping. With the 600M model the segment and context variants differ by less than 2 points everywhere. With the 3.3B model the context variant pulls ahead on ArabicMMLU, by 5.2 points for Qwen2.5-7B and 3.1 for Llama-3.1. The larger model can use discourse context that the smaller one wastes, so the two variants should not be treated as interchangeable once the translator is good enough. The 3.3B output is also slightly shorter: 178.7 prompt tokens against 189.8 on Belebele, so it is both more accurate and cheaper.

E. What the translation actually breaks

Aggregate numbers say the pivot costs accuracy without saying why. We isolated the 107 ArabicMMLU items where Qwen2.5-14B answered correctly from the Arabic and all three open models answered incorrectly from both the 600M and the 3.3B pivot, so the translation is the only thing that changed. Reading them shows four recurring mechanisms. Arabic is glossed in English here rather than set in script.

The task disappears. A grammar item asks which preposition belongs in the gap in "I left [gap] the airport", and the four options are the Arabic prepositions themselves. The 600M model renders the instruction as "choose the right runic" and the options as "B.", "To the …", "Who ?" and "In .". The 3.3B model asks instead for "the appropriate hyphen". No answer is recoverable from either English version, and the item is not a translation failure so much as a question that stops existing outside Arabic.

Proper nouns are translated literally. One item asks where al-Tahtawi enrolled at the age of sixteen, with al-Azhar among the options. The name also means "blossoms", so both translators return "the flowers". In an Islamic Studies item the gate of Paradise named al-Rayyan becomes "Ryan, please" at 600M and "the rhino" at 3.3B. The correct answer is unrecognizable while the question still reads sensibly.

The correct option becomes noise while the distractors stay fluent. "The moon became …" has "full" as its answer. The 600M model returns "I'm going to start" and the 3.3B model "Badra, please", while the three wrong options translate cleanly. All three LLMs then choose a distractor. This is worse for accuracy than a uniformly poor translation, because the surviving options are the only ones that look like answers.

Questions lose content. A proverb item asks what "better than a bad companion" means. The 600M output is just "What does this proverb mean?", with the proverb itself dropped, and it renders the answer "solitude" as "Unit". The 3.3B model keeps the proverb but turns "companion" into "babysitter".

Not every loss is a translation problem. A driving-test item asks what a driver should do before getting into the vehicle; the 3.3B model translates all three options correctly, and all three LLMs still answer wrongly. Items like this one set a floor on what any translator can recover, and they are part of why the 3.3B model closes so little of the gap in Section IV-D.

F. Reasoning mode

Reasoning costs more in Arabic for the open models. On the same questions, step-by-step reasoning written in Arabic is longer than in English: 136.6 against 101.6 tokens for Qwen2.5-7B (1.34×) and 213.4 against 173.7 for Llama-3.1 (1.23×). Since reasoning dominates output, and output is usually priced several times above input, this looked like the larger lever.

GPT-4.1 reverses it. Its Arabic reasoning averages 123.1 output tokens and its English reasoning 135.8, so pivoting makes the output longer while making the prompt shorter. The two cancel: weighted tokens fall by 0.2%, and accuracy falls by 13.2 points, from 96.0 to 83.1 ($p = 2.2 \times 10^{-10}$). In reasoning mode, on this model, the pivot is a loss with no saving attached.

The explanation is in Fig. 1. Qwen and Llama pay 1.63–1.66× for Arabic, so there is inflation to recover; GPT-4o's vocabulary charges 1.38×, and below some threshold the model's own verbosity in English outweighs what

the tokenizer gives back. The argument for reasoning in English holds for high-tax tokenizers and inverts for low-tax ones.

Human English in reasoning mode reaches 98.0 against 96.0 for Arabic, which is 2.0 points and does not reach significance ($p = 0.07$, 7 items gained and 1 lost), while saving 17.3% of weighted tokens. The upper bound itself converges with the baseline as models improve.

G. How much the price ratio matters

Weighted tokens assume output costs 4× input, and that assumption is load-bearing in only one of the two modes. In number-only mode output is 0.3% to 1.3% of all tokens, so the Belebele saving for Qwen2.5-7B moves from 29.3% at a 1:1 ratio to 25.0% at 50:1. Any plausible price ratio gives the same answer. In step-by-step mode output is 30% to 40% of tokens and the saving moves from 24.7% to 9.8% across the same range. Papers that report a single weighted figure for reasoning workloads should state the ratio they assumed.

H. AI-free routing

Can ordinary code decide, per question, when a pivot is safe? We computed five reference-free checks on each translation: length ratio, repeated trigrams, lost numbers, duplicated options and leftover Arabic script. None correlates with translation quality (Spearman ρ between 0.02 and 0.08). A threshold router tuned on even-numbered passages and tested on odd ones does not beat the better fixed strategy.

A sixth check, built to repair rather than to route, fails the same way. It keeps the Arabic option text when a short option expands suspiciously or acquires English pronouns. Items it flags are no harder than items it does not: on ArabicMMLU with Qwen2.5-7B, flagged items score 54.4% and unflagged ones 47.1%. It fires on 85.4% of mathematics questions, one of the least damaged subjects.

Per-question choice has large headroom, since an oracle picking the better strategy for each question would reach 85–90%. Two answer-level signals do separate right from wrong, and both cost more than they save. When the Arabic and pivot answers agree, which happens on 52–82% of items, that answer is right 65–94% of the time, against 55–90% overall; but obtaining the agreement requires both answers, so there is nothing left to save. Agreement among three cheap English prompts separates 87.4% from 42.6% on Belebele and 57.2% from 30.0% on ArabicMMLU, which is a strong signal, yet three prompts at 0.7 the cost of one Arabic prompt come to 2.1×, and a cascade that escalates disagreements to Arabic costs 2.30–2.57× the tokens while losing accuracy in three of four settings. The failures are detectable; they are not detectable cheaply.

What does work without any AI call at inference time is a per-model decision. With a calibration set of 100 parallel questions, the better strategy is chosen in 100% (Llama-3.1, step-by-step), 98% (Qwen2.5-7B, number-only) and 93% (Qwen2.5-7B, step-by-step) of 2,000 resamples. The exception is Llama-3.1 number-only at 73%, where the true gap is 3.4 points and a wrong choice costs little. The calibration questions were drawn from the evaluation pool, and a deployment should hold them out.

I. Time and memory

Fewer tokens is not automatically less time. Local translation with the 600M model took 20.4 ms per sentence, or 0.14 s per Belebele question. Including it, the pivot was slightly faster than asking in Arabic in step-by-step mode (Qwen2.5-7B: 2.31 against 2.42 s per question; Llama-3.1: 4.13 against 4.39 s) but equal or slower in number-only mode (Qwen2.5-7B: 0.32 against 0.25 s). Times are batch-amortized. The 3.3B translator costs 4.9 times as much wall-clock time as the 600M for the gains in Table III.

Memory scales with tokens too. The ArabicMMLU run in Arabic exhausted a 15 GB T4 with 16-question batches for both open models, while the shorter English pivot did not, and the Arabic run had to be repeated with token-budgeted batches.

V. Discussion

When should a deployment pivot? For a model whose Arabic is weak (Llama-3.1 here) and content that originated in English, the pivot is cheaper and more accurate, especially with step-by-step reasoning. For a model strong in Arabic, or for native Arabic content, it saves tokens at a large accuracy cost that grows with how good the model is. A small parallel calibration set, scored once with ordinary code, is enough to make the per-model choice. Self-translation by the LLM should be avoided at every scale we tested.

Route by subject, not by question. The per-subject pattern is stable enough across models to act on. Sending Arabic-language and general-knowledge questions directly and pivoting technical ones captures most of the saving without the worst of the loss, and it needs no per-question estimator, which is the component we could not build cheaply.

Where the saving comes from. With current tokenizers the input saving is bounded near 40% for Qwen and Llama, and we measured 62.6% for Phi-4, whose vocabulary is older and more English-centric. The saving is largest exactly where the tokenizer is worst, so the models that punish Arabic most are the ones where pivoting pays. Reasoning is a second lever for high-tax tokenizers and not a lever at all for GPT-4.1.

What the real fix is not. The human-English bound suggests that a better translator would recover most of the gap. We tested that and it does not: 5.5 times the parameters returns a median 16% of the distance, at five times the translation time. Part of the remaining gap is not a translation problem in the first place, since questions about Arabic itself cannot survive being asked in English. Arabic-aware vocabularies remove the tax entirely, as

ALLaM's 0.97× shows, and they do it without touching accuracy. That is the more promising direction.

VI. Limitations

Every task here is multiple choice, which is the sharpest limit on what we can claim. A deployment that answers open-ended questions must translate the answer back into Arabic, and that second translation can fail in ways a four-option item never exposes: a mistranslated option costs one question, while a mistranslated paragraph of output reaches the user. Back-translation would add local translation time but no paid tokens, so the cost side of our argument should carry over; the accuracy side would need measuring again with generation metrics rather than exact match. We expect the subject pattern to hold, since its cause is the content and not the answer format, but we have not shown it.

Only Modern Standard Arabic is covered. Dialectal Arabic is both more common in real queries and worse served by NLLB, so the pivot's accuracy cost there is likely larger than anything we report, and its token cost is unknown because dialect spellings are less well represented in every vocabulary we measured.

We test five models, two translators and greedy decoding with a single seed. Step-by-step results use 300 questions and cover three of the five models; Phi-4 and Qwen2.5-14B were not run in that mode, so the reasoning-mode finding in Section IV-F rests on one API model. Weighted tokens assume a 4:1 output-to-input price ratio, whose effect we quantify in Section IV-G. ArabicMMLU has no human English version, so there is no upper bound to compare the pivot against on that dataset; the gap-closed column of Table III therefore measures progress toward the Arabic baseline for ArabicMMLU and toward human English for Belebele, and the two halves of that column are not on the same scale. Azure's content filter refused 0.35% of GPT-4.1 prompts and 0.25% of Phi-4 prompts, unevenly across conditions; those items are excluded from paired comparisons. Phi-4 ignored the 16-token output cap on 29.7% of calls, so its output token counts are not comparable with the other models', and our savings for it are computed on prompt tokens only. Dialectal Arabic and open-ended generation, where answers must also be translated back, are not covered; back-translation would add local translation time but no paid tokens.

VII. Conclusion

The Arabic token tax has shrunk to 1.2–1.7× on current general-purpose tokenizers but has not disappeared, and it still reaches 3.04× on a 2024 model. Pivoting through a free local translator cuts 20–63% of prompt tokens, and how much it cuts is predicted by the tokenizer. Whether it preserves accuracy depends on the model's Arabic ability, in a relationship that held without exception across five models: the better the model reads Arabic, the more pivoting costs, from 8.6 to 24.8 points on native Arabic questions. A translator 5.5 times larger recovers a median 16% of the distance to a human reference, so the limit is not mostly the translator. Deployments should decide per model and per subject, with a small calibration set, and evaluations should include native-language content rather than only benchmarks translated from English.

Acknowledgment

Experiments ran on free Kaggle GPUs and Azure AI Foundry student credit. The study idea originated with the author. Experiment code, statistical analysis, figures and a first draft of this paper were produced with the assistance of the generative AI tool Claude [27]. The author ran all experiments and reviewed the final text.

Data and Code Availability

Prompts, per-question results for all five models, analysis scripts and the Kaggle notebooks are at https://github.com/oblhdj/arabic-english-pivot.

References

[1] R. Pope et al., "Efficiently scaling transformer inference," arXiv:2211.05102, 2022.

[2] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. 29th Symp. Operating Syst. Princ. (SOSP), 2023, pp. 611–626.

[3] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in Proc. 54th Annu. Meeting ACL, 2016, pp. 1715–1725.

[4] A. Petrov, E. La Malfa, P. Torr, and A. Bibi, "Language model tokenizers introduce unfairness between languages," in Adv. Neural Inf. Process. Syst. 36 (NeurIPS), 2023, pp. 36 963–36 990.

[5] O. Ahia et al., "Do all languages cost the same? Tokenization in the era of commercial language models," in Proc. EMNLP, 2023, pp. 9904–9923.

[6] A. V. Solatorio, G. Stefanini Vicente, H. Krambeck, and O. Dupriez, "Double jeopardy and climate impact in the use of large language models," arXiv:2410.10665, 2024.

[7] J. M. Lundin et al., "The token tax: Systematic bias in multilingual tokenization," in Proc. 7th Workshop African NLP (AfricaNLP), 2026, pp. 103–112.

[8] J. Etxaniz, G. Azkune, A. Soroa, O. Lopez de Lacalle, and M. Artetxe, "Do multilingual language models think better in English?" in Proc. NAACL, vol. 2, 2024, pp. 550–564.

[9] H. Huang et al., "Not all languages are created equal in LLMs: Improving multilingual capability by cross-lingual-thought prompting," in Findings of EMNLP, 2023, pp. 12 365–12 394.

[10] A. Nag, A. Mukherjee, N. Ganguly, and S. Chakrabarti, "Cost-performance optimization for processing low-resource language tasks using commercial LLMs," in Findings of EMNLP, 2024, pp. 15 681–15 701.

[11] M. S. Bari et al., "ALLaM: Large language models for Arabic and English," arXiv:2407.15390, 2024.

[12] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in Proc. EMNLP, 2023, pp. 13 358–13 376.

[13] T. Han, Z. Wang, C. Fang, S. Zhao, S. Ma, and Z. Chen, "Token-budget-aware LLM reasoning," in Findings of ACL, 2025, pp. 24 842–24 855.

[14] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.

[15] I. Ong et al., "RouteLLM: Learning to route LLMs with preference data," arXiv:2406.18665, 2024.

[16] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, "Large language models as tool makers," arXiv:2305.17126, 2023.

[17] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," arXiv:2304.05128, 2023.
[18] NLLB Team, "No language left behind: Scaling human-centered machine translation," arXiv:2207.04672, 2022.
[19] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in Adv. Neural Inf. Process. Syst. 35 (NeurIPS), 2022, pp. 24 824–24 837.
[20] L. Bandarkar et al., "The Belebele benchmark: A parallel reading comprehension dataset in 122 language variants," in Proc. 62nd Annu. Meeting ACL, 2024, pp. 749–775.
[21] F. Koto et al., "ArabicMMLU: Assessing massive multitask language understanding in Arabic," in Findings of ACL, 2024, pp. 5622–5640.
[22] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.
[23] A. Grattafiori et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024.
[24] "tiktoken (v0.14.0)," OpenAI, accessed Nov. 3, 2026. [Online]. Available: https://github.com/openai/tiktoken
[25] M. Popović, "chrF: Character n-gram F-score for automatic MT evaluation," in Proc. 10th Workshop Statistical Mach. Transl. (WMT), 2015, pp. 392–395.
[26] M. Post, "A call for clarity in reporting BLEU scores," in Proc. 3rd Conf. Mach. Transl. (WMT), 2018, pp. 186–191.
[27] "Claude ([model version]), Anthropic," accessed Nov. 4, 2026. [Online]. Available: https://claude.ai